Agentic Digital Twin for Predictive Maintenance

in Manufacturing Using Machine Learning and Generative AI

Md Abu Yusuf
Vu Quynh Dinh
CS 582 Machine Learning
October 17, 2026

Abstract

The project's novelty is an agentic Manufacturing Copilot added to a machine-learning digital twin. Using the synthetic AI4I 2020 dataset, we compared Random Forest and XGBoost on a stratified 2,000-record test set. Random Forest achieved 91.67% precision, 80.88% recall, and 85.94% F1 for failure detection and serves as the active twin model. The copilot retrieves evaluated readings, runs hypothetical scenarios, and explains tool evidence. Across 60 evaluation tasks repeated three times, GPT-4.1 mini passed 145 of 180 strict execution trials (80.56%). All 30 requested numerical scenario outputs matched the reference. Ambiguous requests exposed frequent selection of unspecified settings. These findings establish an implemented workflow and execution benchmark. Answer quality and physical maintenance benefits require additional evaluation.

1 Introduction

Motivation and business purpose

Unplanned stoppages and unnecessary maintenance interventions motivate a system that helps engineers interpret machine readings before acting. The business purpose is to prioritize inspections, compare hypothetical operating changes, and communicate model evidence through a familiar conversational interface. Our prototype aims to reduce the effort needed to move from measurements to an informed maintenance discussion. Reduced downtime and maintenance cost are intended benefits; they have not been measured in this study.

Related work

Chen et al. review machine learning in predictive-maintenance digital twins [1]. Luo et al. combine twin models and data for CNC tool-life prediction [2]. Matzka studies explainable models and interfaces [3]. These works motivate prediction and explanation in our project. We use Random Forest [4], XGBoost [5], and LangGraph for copilot orchestration [6].

Project objectives and novelty

Our objectives are to classify machine failure, maintain an evaluated virtual state, support natural-language inspection and what-if questions, and evaluate both predictor performance and agent execution. The project novelty is adding an agentic Manufacturing Copilot to the predictive twin. It selects tools for machine state, history, scenarios, and saved model results, then explains the evidence. Random Forest calculates scores; the agent coordinates the workflow. This implemented integration and its evaluation are our contribution, without a claim of a new classifier or a worldwide first.

The scope is one virtual machine with manually entered readings and evaluated snapshots. Live sensors, remaining useful life estimation, and automatic physical control are outside the implementation. Model-based comparisons support maintenance discussion; they do not establish intervention effects.

2 Dataset and Data Preparation

Dataset and exploratory data analysis

AI4I 2020 contains 10,000 synthetic records: 9,661 normal observations and 339 failures (3.39%), with no missing values [7]. Six inputs comprise product quality (L/M/H), air and process temperature (K), speed (rpm), torque (Nm), and tool wear (min). The binary target indicates machine failure.

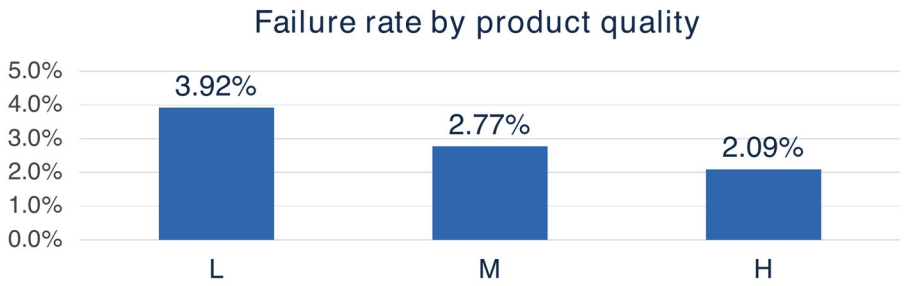


Figure 1 EDA failure rate by product quality in the full dataset [7]

Figure 1 compares within-group failure rates: L 3.92%, M 2.77%, and H 2.09% [7]. L has the highest observed rate; this association does not establish causation. Other EDA uses class counts, boxplots, and torque-speed scatterplots. Failure records show higher median torque and wear, and lower speed. Torque-speed correlation is -0.875; outliers are retained. An always-normal baseline gives 96.61% accuracy and zero failure recall.

Leakage control and one hot encoding

We exclude UDI, Product ID, and failure-mode labels TWF, HDF, PWF, OSF, and RNF to prevent identifier and target leakage. pandas.get_dummies one-hot encodes quality in fixed L/M/H order: L = (1, 0, 0), M = (0, 1, 0), H = (0, 0, 1). All three columns remain when a batch lacks a category; unknown levels are rejected. Training, prediction, and simulation share this encoding, which learns no test-data statistics. Five numeric inputs, six engineered features, and three indicators yield 14 model inputs.

Feature	Definition
Temperature difference	Process temperature minus air temperature
Temperature ratio	Process temperature divided by air temperature
Mechanical power	Torque times angular speed, converted to kW
Normalized torque	Torque divided by speed in thousands of rpm
Wear and torque interaction	Tool wear multiplied by torque
Wear and power interaction	Tool wear multiplied by mechanical power
Product quality indicators	Separate binary indicators for L, M, and H

Table 1 Target-independent engineered features

Power converts rpm to angular speed and watts to kW; feature formulas require no failure label.

Training and test partition

An 80/20 stratified split (seed 42) gives 8,000 training records with 271 failures and 2,000 test records with 68 failures. Both models use identical partitions and feature order, class weighting, and no resampling or learned scaling. Full-dataset EDA preceded this experiment; results represent a fixed-split course benchmark, not independent external validation.

3 Predictor Training and Results

Random Forest uses 300 trees, balanced class weights, and random seed 42. XGBoost uses 300 trees, depth 4, learning rate 0.05, row and column sampling of 0.8, and histogram construction. Its positive-class weight is the training normal-to-failure ratio, 7,729 divided by 271, or 28.52. Both models use a fixed classification threshold of 0.5. The recorded experiment did not perform hyperparameter search or probability calibration.

Metric	Random Forest	XGBoost
Accuracy	99.10%	98.50%
Failure precision	91.67%	75.00%
Failure recall	80.88%	83.82%
Failure F1	85.94%	79.17%
ROC AUC	0.9686	0.9821
Average precision	0.8606	0.8902

Table 2 Saved predictive performance on 2,000 test records

Actual class	Random Forest		XGBoost	
	Predicted normal	Predicted failure	Predicted normal	Predicted failure
Normal (1,932)	1,927 (TN)	5 (FP)	1,913 (TN)	19 (FP)
Failure (68)	13 (FN)	55 (TP)	11 (FN)	57 (TP)

Table 3 Confusion matrices on the same 2,000 test records

Rows are observed classes and columns are predictions; failure is the positive class. TN and TP are correct normal and failure predictions. FP is a false alarm; FN is a missed failure. Precision measures how often failure alerts are correct, recall measures the fraction of failures detected, and F1 balances precision and recall.

Why we selected Random Forest

For this prototype, we prioritize reliable failure alerts and a strong precision-recall balance. Random Forest has higher failure precision (91.67% versus 75.00%) and F1 (85.94% versus 79.17%) at the fixed threshold. Its accuracy is also higher (99.10% versus 98.50%), although accuracy is secondary because failures are rare. The confusion matrices show 5 false alarms for Random Forest versus 19 for XGBoost: 14 fewer false alarms, with 13 missed failures instead of 11. This observed trade-off supports our choice of Random Forest as the active twin model.

XGBoost detects 57 of 68 failures, compared with 55 for Random Forest, and has higher recall (83.82% versus 80.88%), ROC AUC (0.9821 versus 0.9686), and average precision (0.8902 versus 0.8606). The selection therefore reflects our prototype preference, not superiority on every metric or a proven maintenance-cost optimum. Maintenance costs and missed-failure costs were not measured; operational deployment would require validation of the model and threshold against those costs.

Both classifiers use a 0.5 threshold, equivalent to 50 on the displayed risk-score scale. The Failure label is a classifier decision, not independent confirmation of physical breakdown. Scores have not been calibrated as failure probabilities.

4 Digital Twin and Copilot Design

DigitalTwin validates six dashboard inputs and applies the training feature builder. Random Forest calculates a score; the twin stores inputs, classification, indicators, and evaluation time for the copilot.

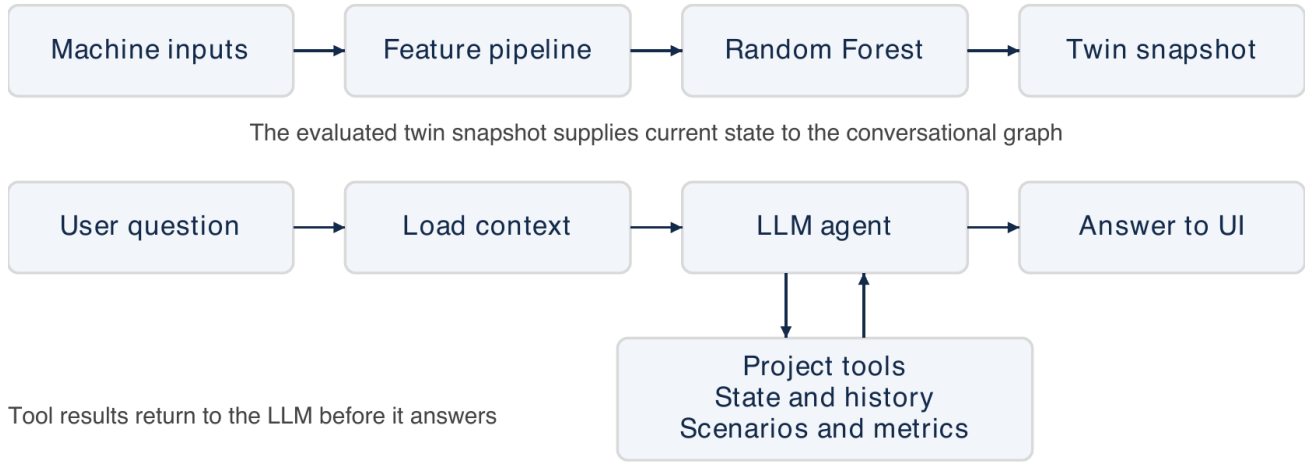


Figure 2 System architecture and conversational execution loop

Figure 2 shows the prediction path and the conversational execution loop. The evaluated twin snapshot supplies current state to the graph, and tool results return to the LLM before it answers.

UpdateState validates and evaluates inputs; PredictFailure returns the prediction; CalculateHealth returns inverse risk. Simulate preserves the baseline; tool replacement changes only wear to zero.

LangGraph nodes and available tools

Three nodes implement the graph: load_context supplies fresh twin context; agent uses GPT-4.1 mini to answer or request tools; tools executes the five functions in Table 4 and returns evidence.

Tool	Evidence or operation
get_current_state	Current evaluated inputs and prediction
get_machine_history	Up to 20 session readings
simulate_scenario	New comparison for requested input changes
get_latest_simulation	Existing comparison for the current baseline
get_model_results	Saved Random Forest or XGBoost test metrics

Table 4 Tools available to the copilot

Fresh context can answer current-risk questions directly. Execution allows six tool rounds and ten completed conversation turns; changing the LLM resets chat. Successful simulations appear in What-if simulation.

Dashboard and Application Output

Figure 3 shows the developed Streamlit application with the Machine state tab selected. This interface displays evaluated model outputs and gives operators access to simulations, saved model results, and the copilot.

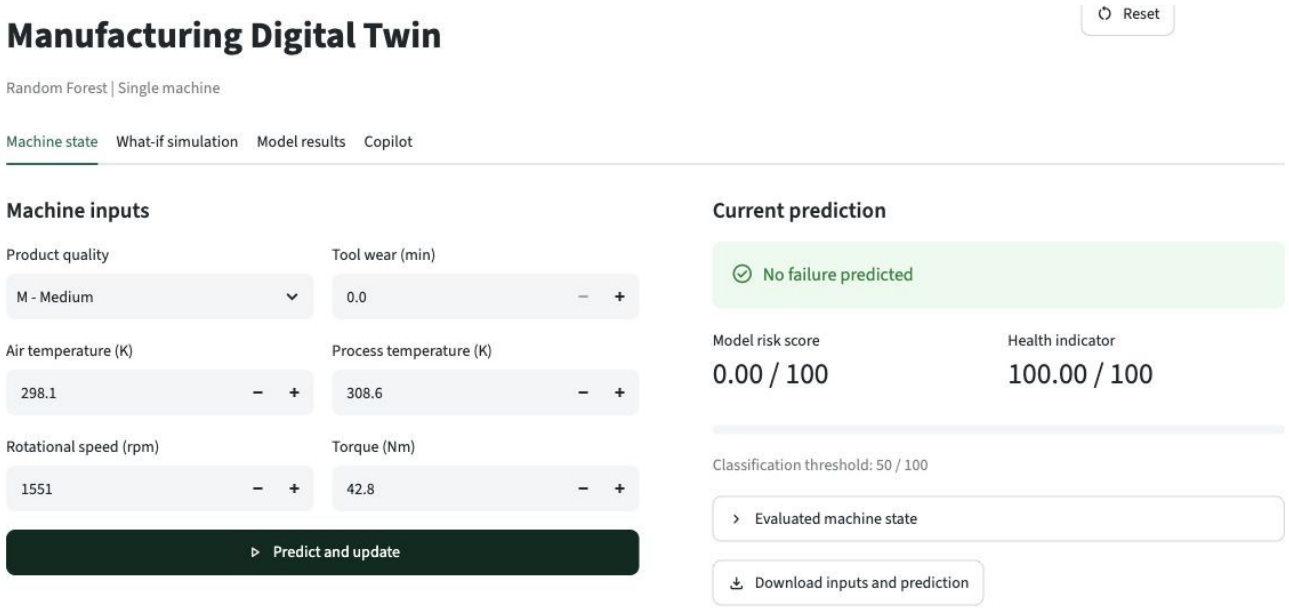


Figure 3 Application dashboard with the Machine state tab selected

Dashboard tabs and operator workflow

- **Machine state.** Enter six readings, predict failure, and update the evaluated state and session history.
- **What-if simulation.** Change selected inputs and compare the scenario with a preserved baseline.
- **Model results.** Review saved classifier metrics and confusion matrices for Random Forest and XGBoost.
- **Copilot.** Select the LLM, ask questions, inspect evidence, and download chat. Record two states for history comparisons; the operator decides whether to act.

Recorded question and recommendation

Operator question What if I replace the tool? Compare baseline and scenario risk.

Tool evidence Wear 154 to 0 min; risk 98.33 to 80.33; both classified Failure.

Recorded recommendation excerpt “I recommend inspecting the tool wear condition and proceeding with replacement if wear is confirmed excessive.”

Figure 4 Recorded question and recommendation excerpt from trial 3

Case EVAL-what_if-01 held other inputs fixed. All repeats passed six checks (3.07, 2.93, 2.64 s). This advice does not establish a physical benefit.

5 Copilot Evaluation and Results

Copilot evaluation protocol

We froze 60 tasks in six categories, separate from 12 development tasks. Distinct states from the ML test partition cover failures, low-score normal records, and records near the threshold. Labels select coverage and never enter prompts. Three repeats with a fresh twin and conversation yield 180 task-trials and 210 turns including history warm-ups. We recorded the gpt-4.1-mini alias, transcripts, tool evidence, checks, and timing.

Direct twin calls and saved metadata provide references. Checks require a nonempty answer, fresh context, preserved baseline, required tools and exact arguments, no unrequested successful simulation, and matching scenario outputs. Every turn must pass. Verifiable outcomes and repeats follow the motivation of tau-bench [8]; our tasks and checks are project-specific.

Evaluation results and interpretation

Task category	Passed / trials	Pass rate
Current-state explanation	30 / 30	100.00%
What-if simulation	20 / 30	66.67%
History and follow-ups	28 / 30	93.33%
Model metric retrieval	30 / 30	100.00%
Ambiguous or invalid requests	10 / 30	33.33%
Unsupported requests	27 / 30	90.00%
Overall	145 / 180	80.56%

Table 5 Strict automatic execution checks

No execution errors occurred. Mean response time was 2.14 seconds per turn, excluding initialization; 46 of 60 tasks (76.67%) passed all repeats. Resampling whole tasks with their three trials gives a descriptive 95% bootstrap interval of 70.56%-89.44% (10,000 resamples, seed 42), conditional on this stress-test suite.

All 30 requested numerical scenario outputs matched references. Strict flags comprise ten redundant-baseline-argument trials, two missing history-tool calls, twenty ambiguous or invalid trials needing clarification or correction, and three guarantee-request trials that simulated replacement while denying a guarantee. These execution checks do not measure hallucination rate or overall answer accuracy.

6 Setup Limitations and Conclusion

Application setup and reproducibility

Install dependencies in a Python virtual environment and launch the dashboard from the project root. XGBoost requires OpenMP on macOS. Configure an OpenAI API key through dashboard settings or the documented environment/secrets mechanism. Local prediction and manual simulation work without an API key.

```
python -m pip install -r requirements.txt
python -m streamlit run app.py
python -m src.evaluation.run --split evaluation --trials 3
```

The four tabs provide machine state, what-if simulation, model results, and copilot chat. Each evaluation saves transcripts, checks, timing, and a manifest in a new directory, preserving the GPT-4.1 mini baseline. Saved artifacts record feature order, parameters, hashes, and package versions.

Limitations and future work

The data are synthetic, and the random split does not establish performance on future sensor streams or other machines. Scores are uncalibrated, and the health indicator subtracts risk from 100 rather than measuring condition independently. Simulated input changes estimate model sensitivity without a physical or causal guarantee. The system provides neither feature attribution nor remaining useful life. It also lacks maintenance-manual retrieval and automatic machine control.

The copilot suite balances categories for stress coverage and is not a representative workload sample. The run records a model alias rather than a pinned snapshot. Independent answer grading, a no-tools LLM comparison, and an operator study are still pending. Next work should enforce explicit scenario values and units, evaluate prompt/tool changes on development cases, calibrate scores on validation data, and collect a fresh final task set before claiming improvement.

Conclusion

The project contributes an agentic Manufacturing Copilot layered on a predictive digital twin. Its novelty within this project is conversational access to machine evidence and model-based what-if comparisons through tool selection and execution. Random Forest achieved 85.94% failure F1, while GPT-4.1 mini passed 80.56% of strict execution trials. The main observed agent weakness was choosing settings for incomplete requests. These results support the prototype and its evaluation method, while leaving answer correctness and maintenance benefits for further study.

References

[1] C. Chen, H. Fu, Y. Zheng, F. Tao, and Y. Liu. The advance of digital twin for predictive maintenance: The role and function of machine learning. Journal of Manufacturing Systems 71, pp. 581-594, 2023. https://doi.org/10.1016/j.jmsy.2023.10.010.

[2] W. Luo, T. Hu, Y. Ye, C. Zhang, and Y. Wei. A hybrid predictive maintenance approach for CNC machine tool driven by Digital Twin. Robotics and Computer-Integrated Manufacturing 65, 101974, 2020. https://doi.org/10.1016/j.rcim.2020.101974.

[3] S. Matzka. Explainable Artificial Intelligence for Predictive Maintenance Applications. AI4I, pp. 69-74, 2020. https://doi.org/10.1109/AI4I49448.2020.00023.

[4] L. Breiman. Random Forests. Machine Learning 45, pp. 5-32, 2001. https://doi.org/10.1023/A:1010933404324.

[5] T. Chen and C. Guestrin. XGBoost: A Scalable Tree Boosting System. KDD, 2016. https://doi.org/10.1145/2939672.2939785.

[6] LangChain. LangGraph overview. https://docs.langchain.com/oss/python/langgraph/overview. Accessed October 17, 2026.

[7] UCI Machine Learning Repository. AI4I 2020 Predictive Maintenance Dataset, 2020. https://doi.org/10.24432/C5HS5C.

[8] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. tau-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains, 2024. https://arxiv.org/abs/2406.12045.